Logic-Flow Code Visualizer (LFCV)

Comparative Report: Earlier vs Enhanced Version

Group Members

NAME	PRN	ROLL NO.
PRAKUL SINGH	1032233410	44
VEDIKA RANA	1032233559	49

T.Y. B.Tech CSE

(Artificial Intelligence & Data Science)

Dr. Vishwanath Karad MIT World Peace University

2025-2026

1. Comparative Summary

The original LFCV version already supported static Java analysis, bulk scanning, integrity checking, and Control Flow Graph rendering. The updated version retains the same dashboard style while adding version-aware comparison and graph-difference visualization.

Feature Area	Earlier Version	Updated Version	Impact
Java File Handling	Each file was analysed as an isolated upload.	Re-upload of the same submission can be tracked as a new version with stored history metadata.	Supports iterative review.
Line-by-Line Comparison	No built-in source comparison between versions.	Dedicated Version Compare view shows unchanged, modified, added, and removed lines.	Improves code review clarity.
CFG Selection	CFG rendering depended mainly on table row selection.	Dedicated CFG submission selector and Show CFG action were added for reliable switching.	More foolproof workflow.
CFG Colour Logic	Single node-type based colour logic only.	Version-aware highlight mode accents added or modified nodes and mutes unchanged nodes.	Makes change hotspots visible.
Version Identity	No visible version label in analysis detail.	Version label, previous version reference, and change counts are displayed in the detail panel.	Adds traceability.
Demonstration Support	No guided version comparison samples.	Ready-to-use sample Java files were added for validating v1 and v2 comparison behaviour.	Faster academic demo preparation.

2. Detailed Enhancements

2.1 Version-Aware Comparative Analysis

- A new comparison service stores submission history and computes differences across versions of the same Java submission.
- The updated system marks code lines as unchanged, modified, added, or removed.
- The Version Compare tab provides an interface similar to conversational code review, making the change list easier to interpret.
- Version history labels are retained so faculty can understand how a submission evolved across uploads.

2.2 Control Flow Graph Enhancements

- The CFG tab now includes a dedicated dropdown and action button so a specific submission can be visualised on demand.
- The baseline graph remains visually consistent with the existing LFCV theme.
- For later versions, only added or modified nodes are accent-coloured, while unchanged nodes are muted.
- The active version palette is displayed in the CFG header, making it easier to verify that version-specific highlighting is applied.

2.3 User Experience Continuity

- The original dashboard structure, dark palette, and reporting-oriented academic look were preserved.
- The enhancements were integrated into the same application rather than presented as a visually separate tool.
- This ensures the updated version still appears as an enhancement to LFCV rather than a completely rebuilt program.

3. Implementation Notes

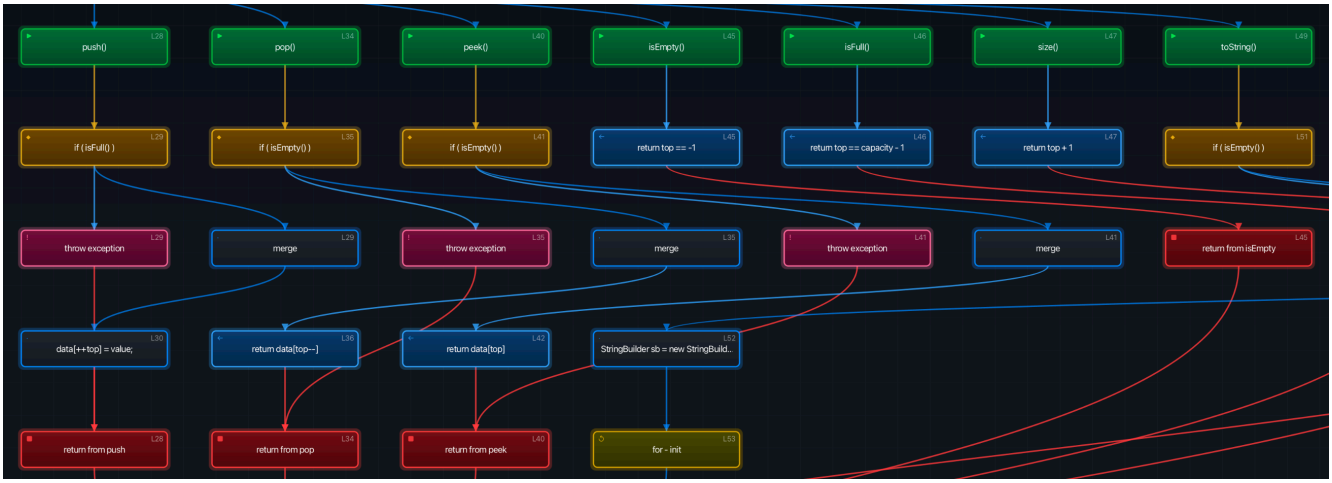
The updated version focuses on the first two requested enhancement areas: comparative analysis of repeated Java uploads and Control Flow Graph highlighting of changes across versions. The report therefore concentrates on these completed and stabilised changes.

During the update, sample Java files were also added to help validate the comparison workflow. This reduces ambiguity during project demonstration and helps verify that the CFG for different versions can be selected and rendered correctly.

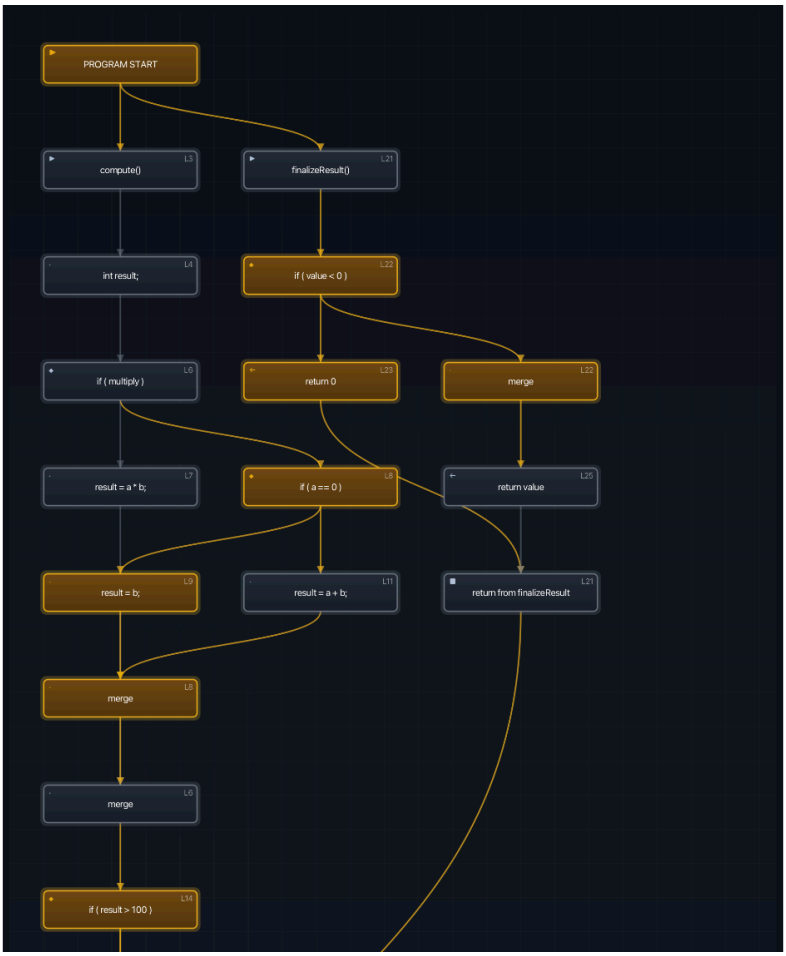
4. Screenshot Placeholders

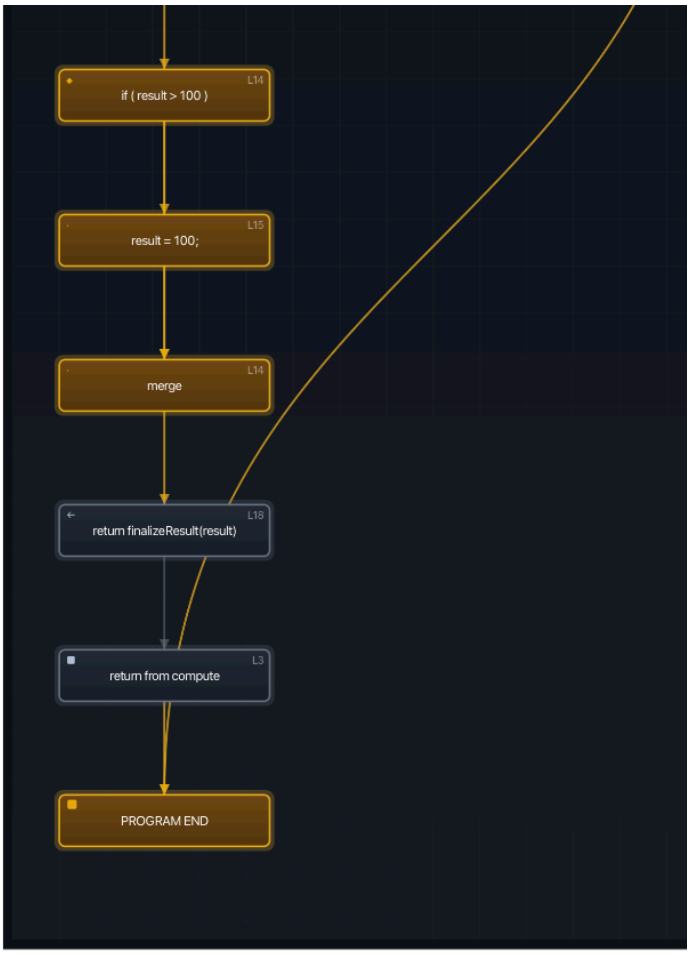
The following placeholders are intentionally left for manual insertion of screenshots by the project team.

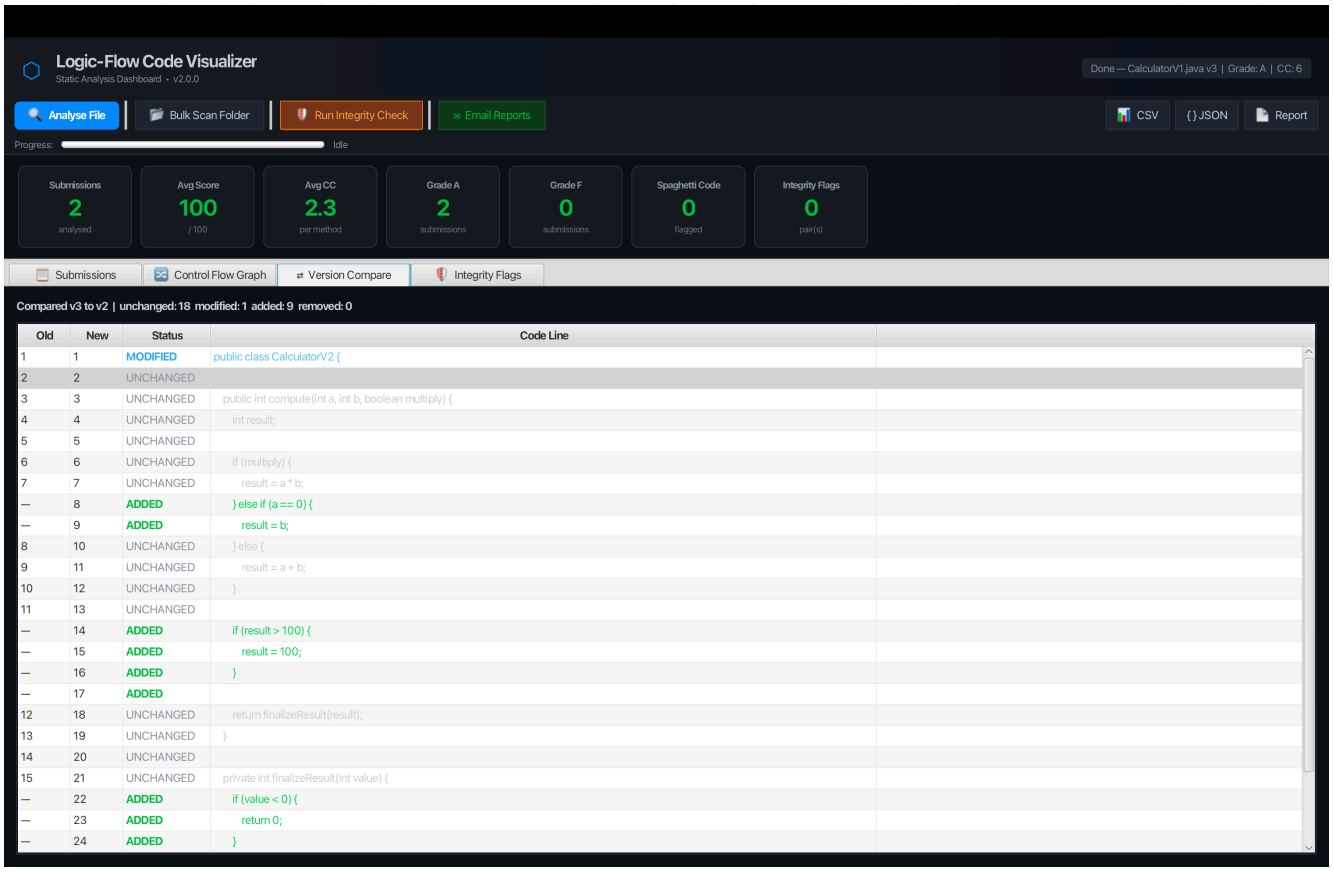
Earlier Version:

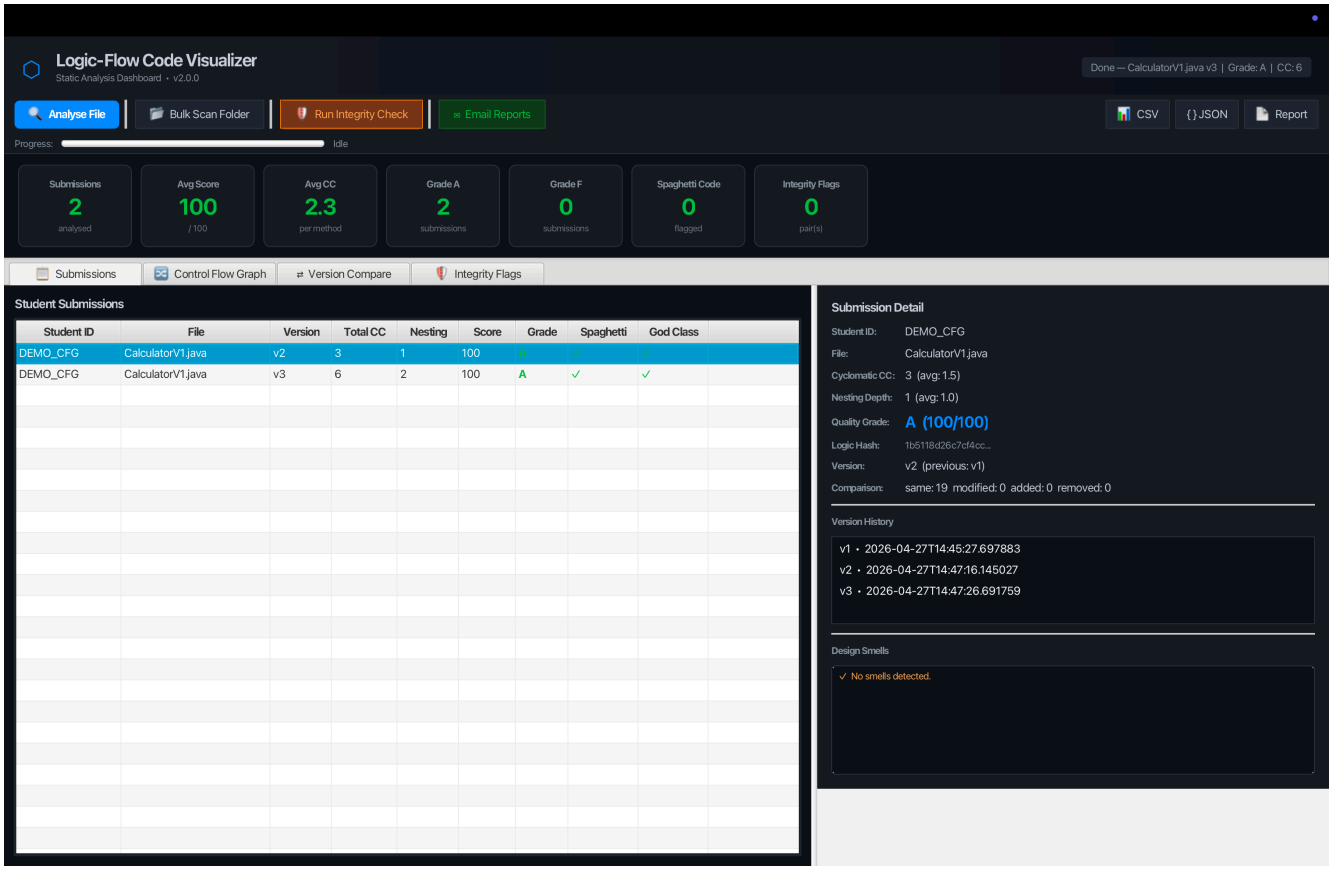


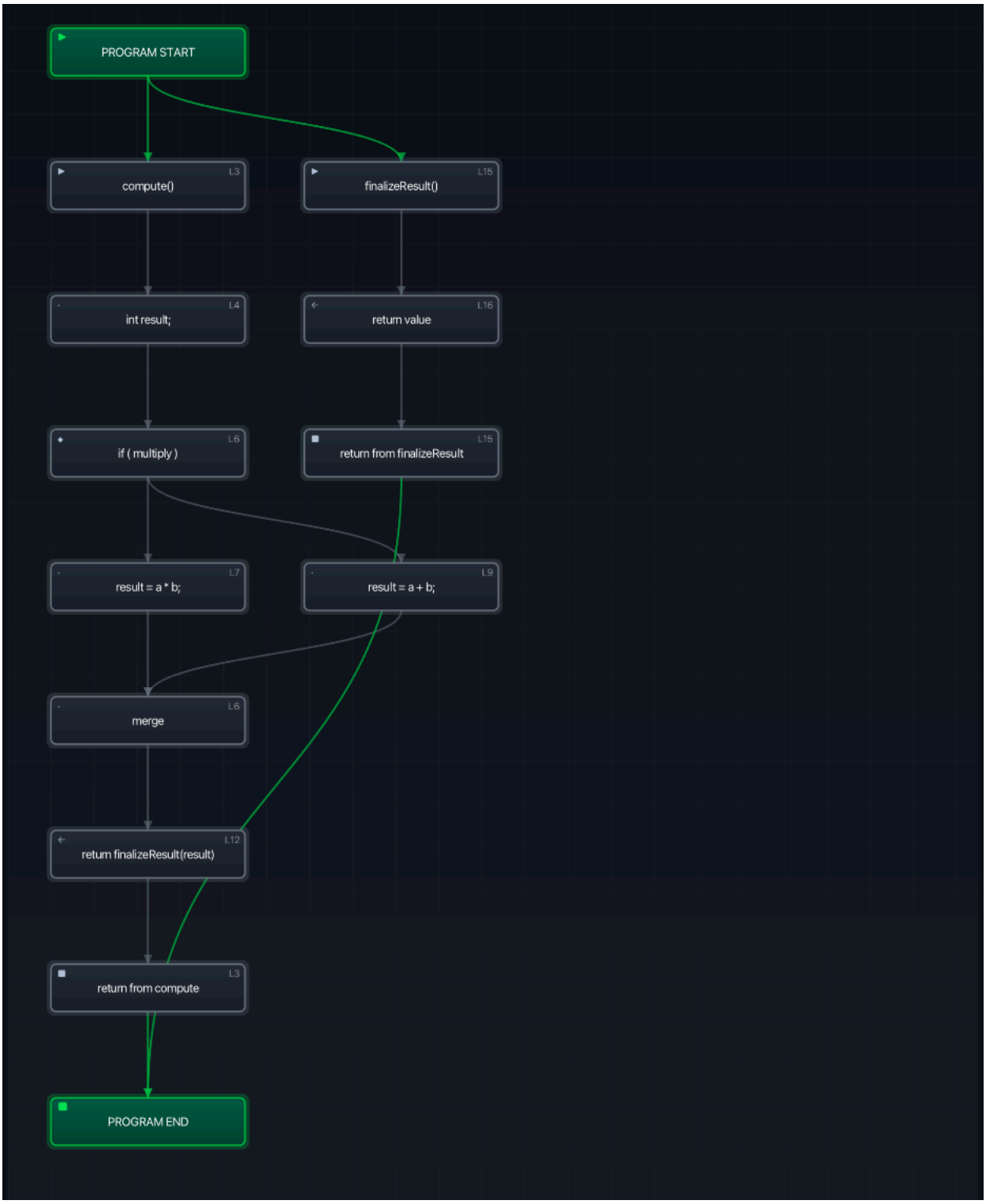
Enhanced Version:






Logic-Flow Code Visualizer
Static Analysis Dashboard • v2.0.0
Done — CalculatorV1.java v3 | Grade: A | CC: 6
Analyse File
Bulk Scan Folder
Run Integrity Check
Email Reports
CSV
{ } JSON
Report
Progress:
Idle
Submissions 2 analysed
Avg Score 100 / 100
Avg CC 2.3 per method
Grade A 2 submissions
Grade F 0 submissions
Spaghetti Code 0 flagged
Integrity Flags 0 pair(s)
Submissions
Control Flow Graph
Version Compare
Integrity Flags
Compared v3 to v2 | unchanged: 18 modified: 1 added: 9 removed: 0
Old New Status Code Line
1 1 MODIFIED public class CalculatorV2 {
2 2 UNCHANGED
3 3 UNCHANGED public int compute(int a, int b, boolean multiply) {
4 4 UNCHANGED int result;
5 5 UNCHANGED
6 6 UNCHANGED if (multiply) {
7 7 UNCHANGED result = a * b;
— 8 ADDED } else if (a == 0) {
— 9 ADDED result = b;
8 10 UNCHANGED } else {
9 11 UNCHANGED result = a + b;
10 12 UNCHANGED }
11 13 UNCHANGED
— 14 ADDED if (result > 100) {
— 15 ADDED result = 100;
— 16 ADDED }
— 17 ADDED
12 18 UNCHANGED return finalizeResult(result);
13 19 UNCHANGED }
14 20 UNCHANGED
15 21 UNCHANGED private int finalizeResult(int value) {
— 22 ADDED if (value < 0) {
— 23 ADDED return 0;
— 24 ADDED }


Logic-Flow Code Visualizer
Static Analysis Dashboard • v2.0.0
Done — CalculatorV1.java v3 | Grade: A | CC: 6
Analyse File
Bulk Scan Folder
Run Integrity Check
Email Reports
CSV
{ } JSON
Report
Progress:
Idle
Submissions 2 analysed
Avg Score 100 / 100
Avg CC 2.3 per method
Grade A 2 submissions
Grade F 0 submissions
Spaghetti Code 0 flagged
Integrity Flags 0 pair(s)
Submissions
Control Flow Graph
Version Compare
Integrity Flags
Student Submissions
Student ID File Version Total CC Nesting Score Grade Spaghetti God Class
DEMO_CFG CalculatorV1.java v2 3 1 100 A ✓ ✓
DEMO_CFG CalculatorV1.java v3 6 2 100 A ✓ ✓
Submission Detail
Student ID: DEMO_CFG
File: CalculatorV1.java
Cyclomatic CC: 3 (avg: 1.5)
Nesting Depth: 1 (avg: 1.0)
Quality Grade: A (100/100)
Logic Hash: 1b5118d26c7cf4cc...
Version: v2 (previous: v1)
Comparison: same: 19 modified: 0 added: 0 removed: 0
Version History
v1 • 2026-04-27T14:45:27.697883
v2 • 2026-04-27T14:47:16.145027
v3 • 2026-04-27T14:47:26.691759
Design Smells
✓ No smells detected.


PROGRAM START
L3
compute()
L15
finalizeResult()
L4
int result;
L16
return value
L6
if (multiply)
L15
return from finalizeResult
L7
result = a * b;
L9
result = a + b;
L6
merge
L12
return finalizeResult(result)
L3
return from compute
PROGRAM END

5. Conclusion

The updated LFCV version extends the earlier academic static analysis tool with practical version comparison and graph-difference visibility. These changes make the system more suitable for iterative code review, project viva demonstrations, and evaluation of evolving Java submissions.

Because the enhancements were applied within the same interface theme and dashboard structure, the final result continues to reflect the original LFCV identity while offering stronger analytical capability.